A. Details of Our DOOM Algorithms

Let $\epsilon_{p,\ell,M}$ denote the probability that a single iteration of `Generic-DOOM` returns a solution $\vec{e}$. The expected runtime is then given by

$$T_{p,\ell,M} \coloneqq \frac{1}{\epsilon_{p,\ell,M}} \cdot \big(T_{\mathrm{Init}}(p,\ell,M) + T_{\mathrm{Enum}}(p,\ell,M)\big),$$

where $T_{\mathrm{Init}}(p,\ell,M)$ and $T_{\mathrm{Enum}}(p,\ell,M)$ are the (expected) runtimes of the initial transformation and the enumeration respectively. To be more precise, when performing our calculations, we give vector additions a cost linear in their length and matrix-vector products a cost equal to the product of the dimensions.

A good metric for the performance of a DOOM algorithm is the $\log_M$ speed-up gained from using M instances:

$$\log_M \left(\frac{T_{p_1,\ell_1,1}}{T_{p_M,\ell_M,M}} \right).$$

For memory, the cost is given by

$$\max\{S_{\mathrm{Init}}(p,\ell,M), S_{\mathrm{Enum}}(p,\ell,M)\},$$

where $S_{\mathrm{Init}}(p,\ell,M)$ and $S_{\mathrm{Enum}}(p,\ell,M)$ denote the memory needed for the initial transformation and enumeration, respectively. We give storing a vector a cost linear in its length.

A.0.1. Concrete Parameter Selection.

All DOOM algorithms are parametrised and we optimise these parameters to get a minimal runtime.

Further, if the computed runtime with a certain amount of instances is higher than the computed runtime with fewer instances, then we ignore the one with more instances. Thus, as a function in the number of instances provided, the runtime will be monotonically decreasing.

A.1. Cost of Initial Transformation

Since the upcoming DOOM algorithms perform the same initial transformation, let us analyse its cost at this point.

The first step of randomly permuting the columns of the parity-check matrix can be done in time n.

For the partial Gaussian elimination, we can perform it simultaneously on the permuted matrix and the syndromes by viewing them as a single matrix:

$$\begin{pmatrix} I_{n-k-\ell} & P_1 & \vec{s}_1^{(1)} \cdots \vec{s}_1^{(M)} \\ 0 & P_2 & \vec{s}_2^{(1)} \cdots \vec{s}_2^{(M)} \end{pmatrix} = G \cdot \begin{pmatrix} P^\pi & \vec{s}^{(1)} \cdots \vec{s}^{(M)} \end{pmatrix}.$$

We perform $n-k-\ell$ steps, and in each step we update an $(n-k) \times (n+M)$ matrix. Thus, the partial Gaussian elimination can be done in time $(n-k-\ell)(n-k)(n+M)$.

This initial transformation is successful if the submatrix formed by the first $n-k-\ell$ rows and columns of the permuted matrix has full rank. The probability for this can be bounded from below by 0.28, cf. [Mor06, section 1.2]. In the computations, we will therefore divide the runtime for this initial transformation by this bound to account for the expected number of repetitions.

Concerning memory, we just need to store the initial parity-check matrix and all syndromes, as all operations can be done in place. So in total, we have

$$T_{\text{Init}}(p,\ell,M) = \frac{1}{0.28}\big(n + (n-k-\ell)(n-k)(n+M)\big) \quad \text{and}$$
$$S_{\text{Init}}(p,\ell,M) = (n-k)(n+M).$$

A.2. DOOM With Dumer-Stern (DS)

In DS-ISD, potential sub-solutions $\vec{e}_2 \in \mathbb{F}_2^{k+\ell}(p)$ are enumerated via Meet-in-the-Middle, or rather a 2-list algorithm. Our DOOM variant is a generalisation of this. For this, we split the vector into two parts $\vec{e}_2 = (\vec{e}_{21}\,\vec{e}_{22})^T$, which allows to transform the syndrome decoding equation in the following way:

$$\begin{aligned} \vec{s}_2^{(i)} &= P_2\vec{e}_2 = P_2(\vec{e}_{21}\,0)^T + P_2(0\,\vec{e}_{22})^T \\ \iff P_2(\vec{e}_{21}\,0)^T &= \vec{s}_2^{(i)} - P_2(0\,\vec{e}_{22})^T. \end{aligned} \tag{1}$$

See Figure 1 for how we do the splitting.

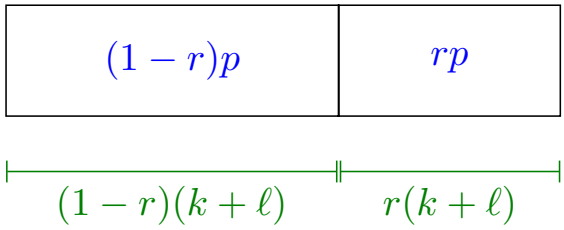

Figure 1: Distribution of the length (green) and weight (blue) of $\vec{e}_2$ in DS-DOOM

Now two lists are filled with each side of Eq.(1), respectively:

$$\begin{aligned} L_1 &:= \Big\{ P_2(\vec{e}_{21}\,0)^T \,:\, \vec{e}_{21} \in \mathbb{F}_2^{(1-r)(k+\ell)}\big((1-r)p\big) \Big\} \\ L_2 &:= \Big\{ \vec{s}_2^{(i)} - P_2(0\,\vec{e}_{22})^T \,:\, i \in \{1,\dots,M\},\, \vec{e}_{22} \in \mathbb{F}_2^{r(k+\ell)}(rp) \Big\}. \end{aligned}$$

The parameter $r \in [0, 1/2]$ allows us to distribute the weight p such that the sizes of the these initial lists are balanced, i.e. $|L_1| \approx |L_2|$.

We perform a matrix-vector multiplication for each element in both lists, and in L_2 we additionally perform a vector addition. Thus, the construction of these lists costs $(|L_1| + |L_2|) \cdot (k+\ell) \cdot \ell + |L_2| \cdot \ell$.

To get the list $L_1 \bowtie L_2$ containing the sub-solutions, we use a hash joining algorithm. For this, the smaller of the two lists is hashed into a hash map, while each element of the other list is computed on the fly and searched for in the hash map. This requires $\min\{|L_1|, |L_2|\} \cdot \ell$ bits of memory. In case of a match, this yields a sub-solution $\vec{e}_2 \coloneqq (\vec{e}_{21}\, \vec{e}_{22})^T$ which can be checked directly whether it also yields a solution to the original problem, i.e. whether $\mathrm{w}(\vec{s}_1^{(i)} - P_1\vec{e}_2) = w - p$. This check involves a matrix-vector multiplication and we get $\vec{e}_2$ by adding the corresponding vectors from the initial lists, so in total the construction of $L_1 \bowtie L_2$ takes time $|L_1 \bowtie L_2| \cdot \big((k+\ell) + (n-k-\ell)\cdot(k+l)\big)$.

Under the assumption that P_2 behaves like a uniformly random linear map, the expected size of the joined list is given by

$$\mathbb{E}|L_1 \bowtie L_2| = \frac{|L_1| \cdot |L_2|}{2^\ell},$$

because the list elements are binary vectors of length ℓ that we fully match.

A.2.1. Expected Cost and Parameter Choice.

The algorithm will be successful if at least one of the error vectors is permuted such that its weight w is distributed correctly. Here that means a weight of $w-p$ in the first $n-k-\ell$ non-zero entries, a weight of $(1-r)p$ in the following $(1-r)(k+\ell)$ entries and a weight of rp in the remaining $r(k+\ell)$ entries. For a fixed error vector, this happens with probability

$$\alpha_{p,\ell,r} \coloneqq \frac{\binom{n-k-\ell}{w-p}\binom{(1-r)(k+\ell)}{(1-r)p}\binom{r(k+\ell)}{rp}}{\binom{n}{w}}.$$

Therefore, a permutation is good for at least one error vector with probability

$$\epsilon_{p,\ell,r,M} \coloneqq 1 - (1-\alpha_{p,\ell,r})^M = M\alpha_{p,\ell,r} + O\big((M\alpha_{p,l,r})^2\big).$$

Since $M\alpha_{p,\ell,r}$ is very small, we can safely approximate by

$$\epsilon_{p,\ell,r,M} \approx \min\{1, M\alpha_{p,\ell,r}\}.$$

The expected runtime of DS-DOOM is then given by

$$\begin{aligned}\frac{1}{\epsilon_{p,\ell,r,M}} \cdot \Big(&T_{\mathrm{Init}}(p,\ell,M) + (|L_1| + |L_2|)\cdot(k+\ell)\cdot\ell + |L_2|\cdot\ell \\ &+ \frac{|L_1|\cdot|L_2|}{2^\ell}\cdot\big((k+\ell) + (n-k-\ell)\cdot(k+l)\big)\Big).\end{aligned} \tag{2}$$

For given p, choosing r such that $|L_1| \approx |L_2|$ is only possible if $M \leq \binom{k+\ell}{p}$ (with equality being the case $r=0$). Since the runtime is dominated by the size of the lists, Eq.(2) is then minimised if we also choose $2^\ell \approx |L_1|$. By computing this for all $p \in [0, w]$, we eventually find the parameters for the minimal runtime.

Concerning the memory cost of the enumeration, we need enough space to hash the smaller of both initial lists and to save the matrices P_1, P_2 along with the transformed syndromes:

$$S_{\mathrm{Enum}}(r,p,\ell,M) = \min\{|L_1|, |L_2|\}\cdot\ell + (n-k)\cdot(k+\ell+M).$$

A.3. DOOM With May-Meurer-Thomae (MMT)

The idea of MMT-ISD is to increase the possibilities of finding a desired sub-solution by introducing representations as sums $\vec{e}_2 = \vec{e}_{21} + \vec{e}_{22}$. The summands have the same lengths but lower weights, and are enumerated using a 2-list approach. Our DOOM variant is a generalisation of this which keeps the initial list sizes balanced. See Figure 2 for the splitting of $\vec{e}_2$.

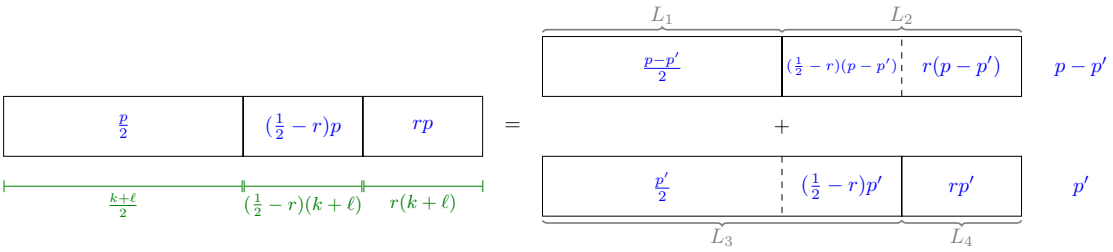

Figure 2: Distribution of the length (green) and weight (blue) in the representations of $\vec{e}_2$ in MMT-DOOM

This allows us to transform the equation for the sub-solutions to

$$P_2(\vec{e}_{211}\,0)^T + P_2(0\,\vec{e}_{212})^T = -P_2(\vec{e}_{221}\,0)^T + \vec{s}_2^{(i)} - P_2(0\,\vec{e}_{222})^T, \tag{3}$$

and fill four lists as follows:

$$L_1 \coloneqq \left\{ P_2(\vec{e}_{211}\,0)^T \,:\, \vec{e}_{211} \in \mathbb{F}_2^{\frac{k+\ell}{2}}(\tfrac{p-p'}{2}) \right\},$$
$$L_2 \coloneqq \left\{ P_2(0\,\vec{e}_{212})^T : \vec{e}_{212} \in \mathbb{F}_2^{(\frac{1}{2}-r)(k+\ell)}\big((\tfrac{1}{2}-r)(p-p')\big) \times \mathbb{F}_2^{r(k+\ell)}\big(r(p-p')\big) \right\},$$
$$L_3 \coloneqq \left\{ -P_2(\vec{e}_{221}\,0)^T \,:\, \vec{e}_{221} \in \mathbb{F}_2^{\frac{k+\ell}{2}}(\tfrac{p'}{2}) \times \mathbb{F}_2^{(\frac{1}{2}-r)(k+\ell)}\big((\tfrac{1}{2}-r)p'\big) \right\},$$
$$L_4 \coloneqq \left\{ \vec{s}_2^{(i)} - P_2(0\,\vec{e}_{222})^T \,:\, i \in \{1,\ldots,M\},\, \vec{e}_{222} \in \mathbb{F}_2^{r(k+\ell)}(rp') \right\}.$$

The parameters $r \in [0, 1/2]$ and $p' \leq p/2$ are used to balance the sizes of these initial lists. Constructing these initial lists has cost analogous to that of Section A.2.

The potential sub-solution $\vec{e}_2$ is then constructed in two steps. First, we match the elements of pairs of neighbouring lists via hash joining. Unlike before, we do not match on all ℓ entries at this step, but just on the first $b \leq \ell$ entries:

$$L_{12} \coloneqq L_1 \bowtie_b L_2 \coloneqq \{P_2\vec{e}_{21} \in L_1 + L_2 \,:\, \pi_b(P_2\vec{e}_{21}) = \vec{0}^b\},$$
$$L_{34} \coloneqq L_3 \bowtie_b L_4 \coloneqq \{\vec{s}_2^{(i)} - P_2\vec{e}_{22} \in L_3 + L_4 \,:\, \pi_b(s_2^{(i)} - P_2\vec{e}_{22}) = \vec{0}^b\},$$

where $\pi_b \colon \mathbb{F}_2^l \to \mathbb{F}_2^b$ is the projection onto the first b entries. So now all pairs of elements of these lists fulfil Eq.(3) in the first b entries. This step costs $(|L_{12}| + |L_{34}|) \cdot (k + \ell)$ in time, as only vector additions are performed.

The second step consists of matching these elements on the remaining $\ell - b$ entries. We achieve this by simply performing a hash join $L_{12} \bowtie L_{34}$. As in DS-DOOM, we can check during the construction of $L \coloneqq L_{12} \bowtie L_{34}$, whether a sub-solution also yields a solution to the original problem. So the cost in time is again $|L| \cdot \big((k + \ell) + (n - k - \ell) \cdot (k + \ell)\big)$.

A.3.1. Expected Cost and Parameter Choice.

The algorithm will be successful if at least one error vector is well-permuted and any of their representations reaches the lists L_{12} and L_{34}. Here, well-permuted means that $w - p$ non-zero entries are distributed in the first $n - k - \ell$ entries, and the remaining p non-zero entries are distributed in the last $k + \ell$ entries according to (the left-hand side of) Figure 2. For a fixed error vector this happens with probability

$$\alpha_{p,\ell,r} \coloneqq \frac{\binom{n-k-\ell}{w-p}\binom{(k+\ell)/2}{p/2}\binom{(1/2-r)(k+\ell)}{(1/2-r)p}\binom{r(k+\ell)}{rp}}{\binom{n}{w}}.$$

Now, for a well-permuted error vector, the number of representations across the initial lists is given by

$$R(p, p', r) \coloneqq \binom{p/2}{p'/2}\binom{(1/2-r)p}{(1/2-r)p'}\binom{rp}{rp'},$$

and the probability that a fixed representation reaches the lists L_{12} and L_{34} is 2^{-b}. Analogous to [MMT11, Theorem 2], one can show that the probability that at least one representation of any well-permuted error vector reaches the lists L_{12} and L_{34} is at least $1/2$ if one chooses

$$b \leq \begin{cases} \log_2(\tilde{M} \cdot R(p, p', r)) - 2 & \text{, for } M = 1 \\ \log_2(\tilde{M} \cdot R(p, p', r)) - 1 & \text{, for } M > 1 \end{cases},$$

where $\tilde{M} \coloneqq \max\{1, M\alpha_{p,\ell,r}\}$ is the expected number of well-permuted error vectors, given at least one.

Analogous to DS-DOOM, the probability that at least one error vector is well-permuted can then be safely approximated by

$$\epsilon_{p,l,r,M} \approx \min\left\{1, \frac{M\alpha_{p,\ell,r}}{2}\right\}.$$

The expected runtime of the enumeration step is again dominated by the sum of the

list sizes, which are the following:

$$
\begin{aligned}
|L_1| &= \binom{(k+\ell)/2}{(p-p')/2}, & |L_2| &= \binom{(1/2-r)(k+\ell)}{(1/2-r)(p-p')}\binom{r(k+\ell)}{r(p-p')},\\
|L_3| &= \binom{(k+\ell)/2}{p'/2}\binom{(1/2-r)(k+\ell)}{(1/2-r)p'}, & |L_4| &= M\cdot\binom{r(k+\ell)}{rp'},\\
\mathbb{E}|L_{12}| &= |L_1|\cdot|L_2|\cdot 2^{-b}, & \mathbb{E}|L_{34}| &= |L_3|\cdot|L_4|\cdot 2^{-b},\\
\mathbb{E}|L| &= |L_{12}|\cdot|L_{34}|\cdot 2^{-(\ell-b)},\\
&\approx |L_1|\cdot|L_2|\cdot|L_3|\cdot|L_4|\cdot 2^{-\ell-b}.
\end{aligned}
$$

Note that the sizes of the last three lists is minimised if we choose b maximally.

To have balanced initial lists, we need that

$$
M \leq \sqrt[3]{\binom{(k+\ell)/2}{(p-p')/2}^2\binom{(k+\ell)/2}{p'/2}^2} = \left(\binom{(k+\ell)/2}{(p-p')/2}\binom{(k+\ell)/2}{p'/2}\right)^{\frac{2}{3}},
$$

with equality occurring (asymptotically) if L_4 only contains the syndromes (i.e. $r=0$), and p' is chosen such that the first three lists are balanced. Under this condition, we then compute the parameters p' and r as follows:

1. Assuming that L_3 and L_4 are balanced, compute p' that minimises the difference of their asymptotic size to the first two lists:

$$
p' \coloneqq \operatorname*{arg\,min}_{0\leq p'\leq p/2}\left|\binom{(k+\ell)/2}{(p-p')/2} - \sqrt{M\cdot\binom{k+\ell}{p'}}\right|.
$$

2. With this p', compute the distribution factor r that actually balances the list sizes of L_3 and L_4:

$$
r \coloneqq \operatorname*{arg\,min}_{0\leq r\leq 1/2}\left|\binom{(k+\ell)/2}{p'/2}\binom{(1/2-r)(k+\ell)}{(1/2-r)p'} - M\cdot\binom{r(k+\ell)}{rp'}\right|.
$$

Concerning the memory cost for the enumeration, a streaming implementation as described in [EMZ22] should be used. The memory cost up to constant factors will then be

$$
\ell\cdot\left(\max\left\{\min\{|L_1|,|L_2|\},\min\{|L_3|,|L_4|\}\right\} + \min\{|L_{12}|,|L_{34}|\}\right),
$$

in addition to the $(n-k)(k+\ell+M)$ bits needed to store the matrices P_1, P_2 and the transformed syndromes.

References

[EMZ22] Andre Esser, Alexander May, and Floyd Zweydinger. McEliece needs a break - solving McEliece-1284 and quasi-cyclic-2918 with modern ISD. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 433–457. Springer, Cham, May / June 2022.

[MMT11] Alexander May, Alexander Meurer, and Enrico Thomae. Decoding random linear codes in $\tilde{\mathcal{O}}(2^{0.054n})$. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 107–124. Springer, Berlin, Heidelberg, December 2011.

[Mor06] Kent E. Morrison. Integer sequences and matrices over finite fields. *Journal of Integer Sequences*, 9, 2006. Article 06.2.1.